

Mini Photoshop: A MATLAB-Based Image Editor

Francesco Albano

Student ID: LJ2506219

School of Computer Science

Beihang University (BUAA)

Beijing, China

Email: francescoalbano@buaa.edu.cn, francescoalbano357@gmail.com

Abstract—This report presents the development of Mini Photoshop, a graphical image editing application built using MATLAB. The application provides essential image manipulation features including brightness, contrast, and saturation adjustments, various filters, geometric transformations, and histogram analysis. Additional features include an RGB curve editor with multi-channel support, automatic filter persistence, EXIF metadata extraction, fullscreen mode, loading progress indicators, separate information windows, and keyboard shortcuts. The project demonstrates modular programming principles, user interface design, and integration of MATLAB's Image Processing Toolbox [2]. The application follows a modular architecture with separate manager classes for better organization and maintainability.

Index Terms—Image processing, MATLAB GUI programming, RGB curves, color correction, filters, histogram equalization, modular architecture, EXIF metadata, curve manipulation, fullscreen mode, keyboard shortcuts, user interface design

I. INTRODUCTION

Digital image editing has become an essential tool in various fields including photography, graphic design, and scientific research. MATLAB provides powerful image processing capabilities through its Image Processing Toolbox [2], making it an ideal platform for developing educational image editing applications.

Mini Photoshop was developed as a learning project to demonstrate the integration of MATLAB's graphical user interface capabilities [3] with image processing algorithms. The application offers a user-friendly interface for performing common image editing operations while maintaining a modular code structure for easy maintenance and extension. For personal interest, the application includes detailed EXIF metadata extraction, allowing users to view comprehensive camera settings, lens information, and other technical details of photographs.

II. DESIGN AND IMPLEMENTATION

A. Architecture Overview

The application follows a modular design pattern with separate manager classes for better separation of concerns:

- **Data Management:** Classes handling data persistence and state (HistoryManager, MetadataExtractor, CurveManager, FilterManager, ImageIOManager)
- **User Interface:** GUI components and visualization (UIManager, WindowManager, main application interface)

- **Business Logic:** Image processing algorithms and event handling (Mini_Photoshop.m, ImageAdjuster.m, ImageFilter.m, PlotManager.m)

The modular design includes separate classes handling different aspects of functionality:

- **Mini_Photoshop.m:** Main application class coordinating event handlers and manager classes
- **ImageAdjuster.m:** Static methods for brightness, contrast, saturation, and gamma corrections
- **ImageFilter.m:** Implementation of various image filters including blur, sharpen, edge detection, and noise reduction
- **HistoryManager.m:** Undo/redo functionality with operation logging
- **PlotManager.m:** Histogram and curve plotting utilities
- **MetadataExtractor.m:** EXIF data extraction from image files
- **CurveManager.m:** Advanced RGB curve management with point manipulation and channel selection
- **FilterManager.m:** Filter state management and automatic reapplication
- **ImageIOManager.m:** Handles image loading, saving, and format management
- **UIManager.m:** Manages user interface components and layout
- **WindowManager.m:** Controls window states, including fullscreen mode and separate information windows

B. Architecture Visualization

The modular architecture of Mini Photoshop is visualized in the class diagram shown in Figure 1. The system follows a clear separation of concerns organized into three main packages:

Core Application: Contains the main GUI class (Mini_Photoshop) and dedicated manager classes for state management (HistoryManager, CurveManager, FilterManager, PlotManager, ImageIOManager, UIManager, WindowManager)

Image Processing: Provides static methods for image adjustments (ImageAdjuster) and filter applications (ImageFilter)

Utilities: Handles metadata extraction (MetadataExtractor)

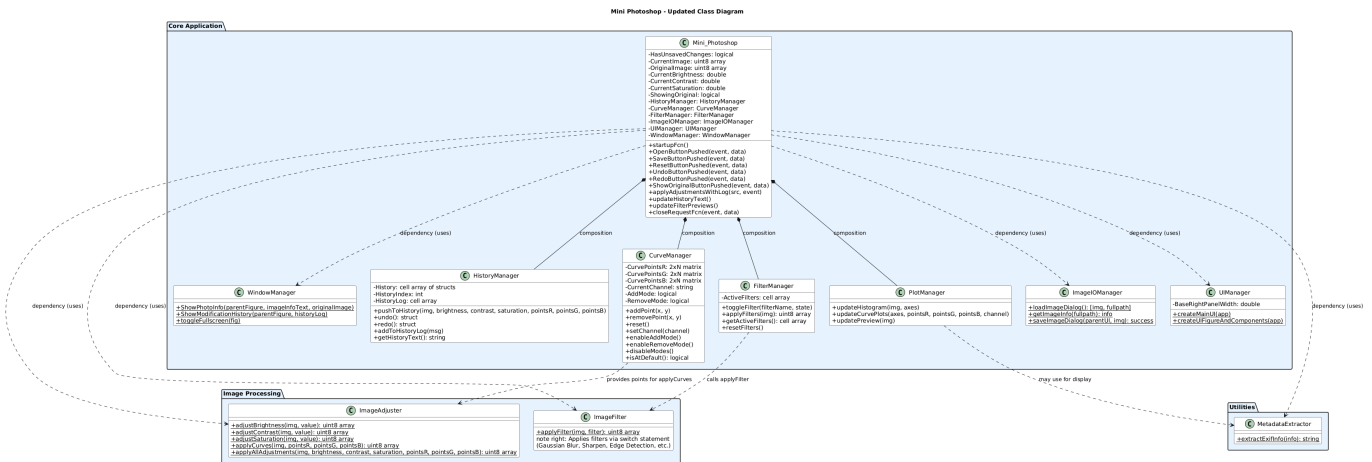


Fig. 1: UML Class Diagram showing the modular architecture of Mini Photoshop

The solid diamond arrows indicate composition relationships, where `Mini_Photoshop` owns and manages the lifetime of the manager classes. The dashed arrows represent usage dependencies, where methods from static utility classes are invoked as needed.

C. User Interface Design

The GUI was implemented using MATLAB's object-oriented programming [3], with all interface elements coded manually for precise control and customization.

- Main image display area with zoom and fit capabilities
- Scrollable interface for modifications organized by workflow: Adjustments, Curves, Filters, Info, History
- Real-time histogram visualization
- Interactive sliders for real-time parameter adjustment
- Advanced RGB curve editor with point manipulation
- Operation history with detailed logging and export functionality
- Unsaved changes detection with confirmation dialog on exit
- Fullscreen mode for immersive editing experience
- Loading progress indicator during image operations
- Separate information windows for image metadata and modification history
- Keyboard shortcuts for common operations

III. KEY FEATURES

A. Image Adjustments

1) *Basic Adjustments*: Brightness, contrast, and saturation adjustments are implemented using HSV color space transformations.

2) *Geometric Transformations:* Rotation (90° , 180° , 270°) and flipping (horizontal/vertical) operations are implemented using MATLAB's `imrotate` and `matrix` operations.

3) *RGB Curve Management*: The application features an RGB curve editing system that provides color correction and gamma control capabilities. The CurveManager class handles all curve-related operations with point manipulation.

Key Features:

- **Multi-Channel Support:** Users can work on individual color channels (Red, Green, Blue) or simultaneously adjust all three channels (RGB mode)
- **Interactive Point Manipulation:** Click-to-add and click-to-remove control points directly on the curve graph
- **Visual Feedback:** Real-time curve plotting with different colors for each channel
- **Modal Operation:** Dedicated Add Point and Remove Point modes for precise control
- **Curve Reset:** One-click reset to default linear curves for all channels
- **Smooth Interpolation:** PCHIP-based curve interpolation for natural tonal transitions

Technical Implementation: The curve system uses a point-based approach where each curve is defined by control points stored in matrices. The CurveManager maintains separate point collections for each RGB channel, allowing independent manipulation while preserving the ability to work on all channels simultaneously. Point coordinates are clamped to valid ranges (0-255) and smooth PCHIP interpolation ensures natural curve shapes.

Integration with Adjustments Pipeline: Curves are applied after brightness and contrast adjustments but before saturation, ensuring proper color correction workflow. The system includes automatic filter reapplication when curves are modified, maintaining consistency in the adjustment pipeline.

History Logging: Detailed operation logging tracks curve modifications with specific channel information:

- "RGB curve point added at (x, y)" for multi-channel edits
- "Red/Green/Blue curve point added at (x, y)" for single channel edits
- "RGB/Red/Green/Blue curve point removed" for point deletions
- "Curves reset" for complete curve restoration

4) *Filter System*: The application supports multiple filters with an intelligent FilterManager that ensures filter persistence

through all adjustment operations. Filters are applied in a fixed sequential order to maintain consistent results:

- 1) Gaussian blur
- 2) Sharpening
- 3) Edge detection (Sobel algorithm)
- 4) Edge detection (Canny algorithm)
- 5) Emboss effect
- 6) Histogram equalization (automatic)
- 7) Histogram equalization (adaptive)
- 8) Noise reduction using median filtering

Automatic Filter Reapplication: When users modify curves or other adjustments after applying filters, the FilterManager automatically reapplies selected filters to maintain visual consistency. This ensures that filter effects persist through the entire editing workflow, eliminating the need for manual reapplication.

Filter Preview System: Real-time thumbnail previews show the effect of each filter on the current image state, allowing users to make informed selections before applying filters to the full-resolution image.

B. Multi-Format Saving

Users can save edited images in various formats including JPEG, PNG, BMP, TIFF, and GIF, providing flexibility for different use cases.

C. Original Image Comparison

The application provides a "Show Original" feature that allows users to instantly view the unmodified original image for direct comparison with the current edited version. This facilitates precise adjustment decisions and ensures the desired modifications are achieved.

D. History Management

The application features a comprehensive undo/redo system implemented through the HistoryManager class, designed to provide robust operation tracking and reversal capabilities:

- **Unlimited Undo/Redo:** Full operation history with forward and backward navigation through all image modifications
- **Operation Logging:** Detailed logging of all image modifications with timestamps and specific operation details
- **Selective Logging:** Prevents duplicate entries while maintaining operation traceability

Technical Implementation: The undo/redo system uses a command pattern approach where each image modification is encapsulated as a reversible operation. The HistoryManager maintains two stacks: one for undo operations and one for redo operations. When a new operation is performed, it is added to the undo stack, and the redo stack is cleared to prevent branching histories.

Key Features:

- **State Preservation:** Each operation stores the complete image state along with adjustment parameters to ensure perfect restoration

- **Memory Usage:** The system stores full image snapshots for each operation, providing unlimited undo capability at the cost of memory consumption
- **Operation Categories:** Different types of operations (adjustments, filters, curves) are logged with specific meta-data for better traceability

Operation Logging Examples: The system logs detailed information for various operations:

- "Brightness adjusted from 0 to +20" with timestamp
- "Gaussian Blur filter applied"
- "RGB curve point added at (128, 150)" with channel specification
- "Image rotated 90° clockwise"

This detailed logging allows users not only to undo operations but also to review the complete editing workflow, making the application suitable for image editing tasks where operation traceability is crucial.

E. Exit Protection

To prevent accidental data loss, the application includes exit protection that tracks modification state and prompts for save confirmation when attempting to exit with unsaved changes.

F. Performance Features

The application provides real-time preview functionality, offering immediate visual feedback for all adjustments to enhance the user editing experience.

G. EXIF Metadata Extraction

For personal interest in photography details, the application extracts and displays EXIF information including camera make/model, lens details, focal length, aperture, shutter speed, ISO settings, and other technical parameters. This allows users to analyze the exact shooting conditions and equipment used for each photograph.

H. Fullscreen Mode

The application supports fullscreen mode for an immersive editing experience, allowing users to maximize the workspace and focus entirely on the image editing process without distractions from the operating system interface.

I. Loading Progress Indicator

During image loading and processing operations, the application displays a simple "Loading..." message to provide visual feedback on the current operation status. This approach informs the user that the application is busy, even though it does not implement a true progress indicator.

J. Separate Information Windows

The application provides dedicated small GUI windows for displaying image information and modification history. These windows update automatically in real-time as changes are made, providing instant access to current metadata and operation logs without cluttering the main interface.

K. Keyboard Shortcuts

For improved efficiency, the application supports keyboard shortcuts for common operations, allowing users to perform actions quickly without navigating through menus or clicking buttons.

IV. TESTING AND VALIDATION

A. Functionality Testing

Each feature was tested individually:

- Image loading and display across different formats (JPEG, PNG, BMP, TIFF, GIF)
- Adjustment sliders response and real-time preview
- RGB curve adjustments for individual color channels
- Filter application and combination effects
- Multi-format saving functionality
- History operations and log export functionality
- EXIF metadata extraction and display
- Unsaved changes detection and exit confirmation

Additionally, combinations of features were tested to ensure proper interaction between adjustments, filters, curves, and transformations.

B. User Interface Testing

The GUI was evaluated for:

- Responsiveness and layout consistency
- Error handling for invalid operations
- Platform compatibility (tested on macOS and Windows)

C. Compatibility Testing

The application was developed using MATLAB R2025b on macOS and was successfully tested on both macOS and Windows platforms. Compatibility was also verified with MATLAB R2022a to ensure broader usability across different MATLAB versions.

D. Performance Considerations

The application provides real-time preview updates for all operations. EXIF extraction is performed only once during image loading to minimize processing overhead.

V. USER GUIDE

This section provides a brief guide on how to use Mini Photoshop.

A. Loading an Image

- 1) Launch the application by running `Mini_Photoshop.m` in MATLAB.
- 2) Click on File → Load Image.
- 3) Select an image file (supported formats: JPEG, PNG, BMP, TIFF, GIF).
- 4) The image will be displayed in the main area with fit options; use MATLAB's native zoom tools for magnification.

B. Applying Adjustments

- 1) Scroll to the Adjustments section.
- 2) Use the sliders for Brightness, Contrast, and Saturation.
- 3) Adjust Gamma correction if needed.
- 4) Changes are applied in real-time; use Undo/Redo if necessary.

C. Using Filters

- 1) Scroll to the Filters section.
- 2) Select one or more filters by checking the corresponding checkboxes (e.g., Gaussian Blur, Sharpen, Edge Detection).
- 3) The selected filters will be applied sequentially in a predefined order when you modify other settings.

D. Managing RGB Curves

- 1) Scroll to the Curves section, select the desired channel from the dropdown: R (Red), G (Green), B (Blue), or RGB (all channels simultaneously).
- 2) To add a control point:
 - a) Click Add Point to enter add mode (button turns green)
 - b) Click directly on the curve graph at the desired position
 - c) The point will be added to the selected channel(s) and the curve will update automatically
- 3) To remove a control point:
 - a) Click Remove Point to enter remove mode (button turns red)
 - b) Click on an existing point on the curve to remove it
- 4) Click Reset Curves to return all channels to their default linear state
- 5) All curve modifications are logged in the History section with specific channel information
- 6) Applied filters are automatically reapplied when curves are modified to maintain visual consistency

E. Viewing Metadata

- 1) Scroll to the Info section.
- 2) View EXIF data including camera settings, lens, aperture, shutter speed, ISO, etc.
- 3) Alternatively, open a separate information window by selecting View → Image Info for a dedicated, auto-updating display.

F. Viewing Modification History

- 1) Scroll to the History section to see the list of applied operations.
- 2) Alternatively, open a separate history window by selecting View → Modification History for a dedicated, auto-updating display.

G. Fullscreen Mode

Fullscreen mode can be activated using your operating system's standard window controls (for example, the green button on macOS). Keyboard shortcuts like F11 may be available on some systems.

H. Keyboard Shortcuts

The application supports the following keyboard shortcuts for quick access to common operations:

- Ctrl+O: Load Image
- Ctrl+S: Save Image
- Ctrl+Z: Undo
- Ctrl+Y: Redo
- Ctrl+I: Open Image Info Window
- Ctrl+H: Open Modification History Window

I. Saving Images

- 1) Click File → Save As.
- 2) Choose format (JPEG, PNG, BMP, TIFF, GIF).
- 3) Specify the file path and save.

J. Exiting the Application

- 1) If there are unsaved changes, a confirmation dialog will appear.
- 2) Click Yes to exit without saving, No to cancel, or Cancel to return.

VI. CONCLUSION

Mini Photoshop demonstrates the development of a functional image editing application using MATLAB, incorporating modular design principles for better organization. The application provides essential image editing features suitable for basic photo manipulation tasks.

One of the key strengths of the application is its comprehensive undo/redo system, which allows users to freely experiment with edits and revert changes at any stage. This feature greatly enhances usability and reliability, making the editing process more flexible and user-friendly.

The project highlights the use of MATLAB's GUI programming capabilities and the Image Processing Toolbox for implementing image editing algorithms. Features like RGB curve control, filter application, and EXIF metadata extraction provide useful functionality for basic image editing.

The development process applied software engineering concepts including modular design, user interface development, and testing methodologies in a practical image processing context.

VII. OPEN SOURCE AVAILABILITY

The complete source code for Mini Photoshop is available as an open source project on GitHub at https://github.com/fra2404/Mini_Photoshop. The repository includes:

- Complete MATLAB source code with modular architecture
- Documentation (this report)
- README with basic usage instructions

ACKNOWLEDGMENT

This project was developed as part of the Technical Computing course requirements.

REFERENCES

- [1] MATLAB Documentation. <https://www.mathworks.com/help/matlab/>
- [2] Image Processing Toolbox User's Guide. MathWorks, 2023.
- [3] MATLAB GUI Development Documentation. https://www.mathworks.com/help/matlab/creating_guis.html

APPENDIX

This appendix contains screenshots of the Mini Photoshop application in use, demonstrating its various features and interface components. All screenshots were taken using a photograph captured by the author, showing the application working with real image data. The following images illustrate the main interface, curve editing capabilities, filter application, information display, and user interaction dialogs.

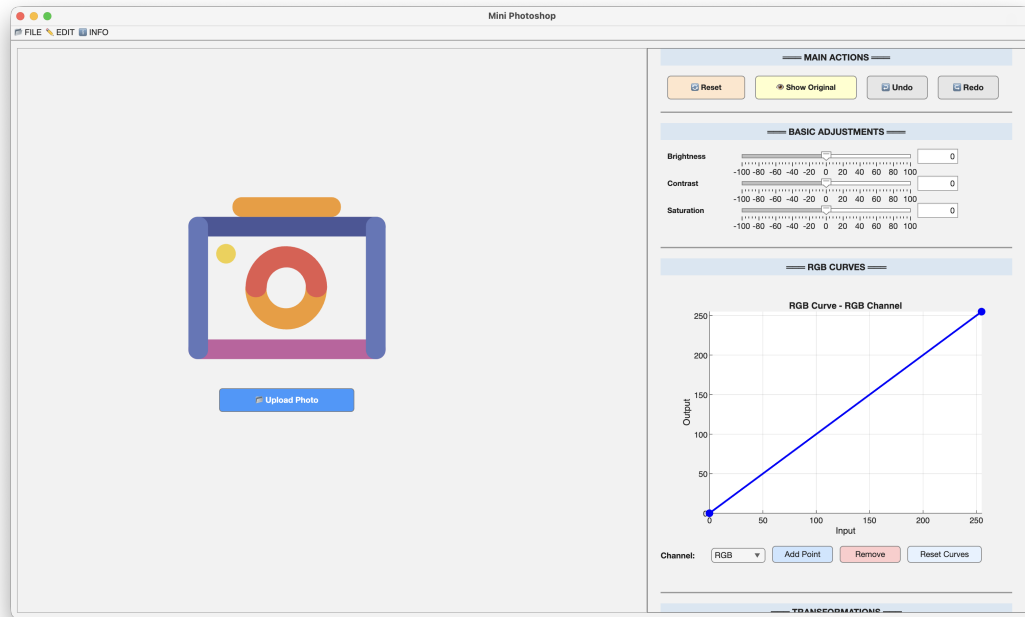
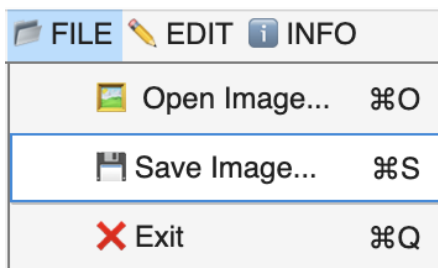
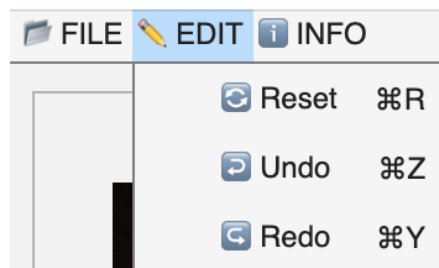


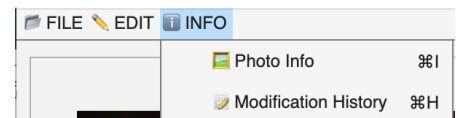
Fig. 2: Main interface of Mini Photoshop showing the image display area and tool tabs.



(a) File menu



(b) Edit menu



(c) Info menu

Fig. 3: Screenshot of the three main menus: File, Edit, and Info.

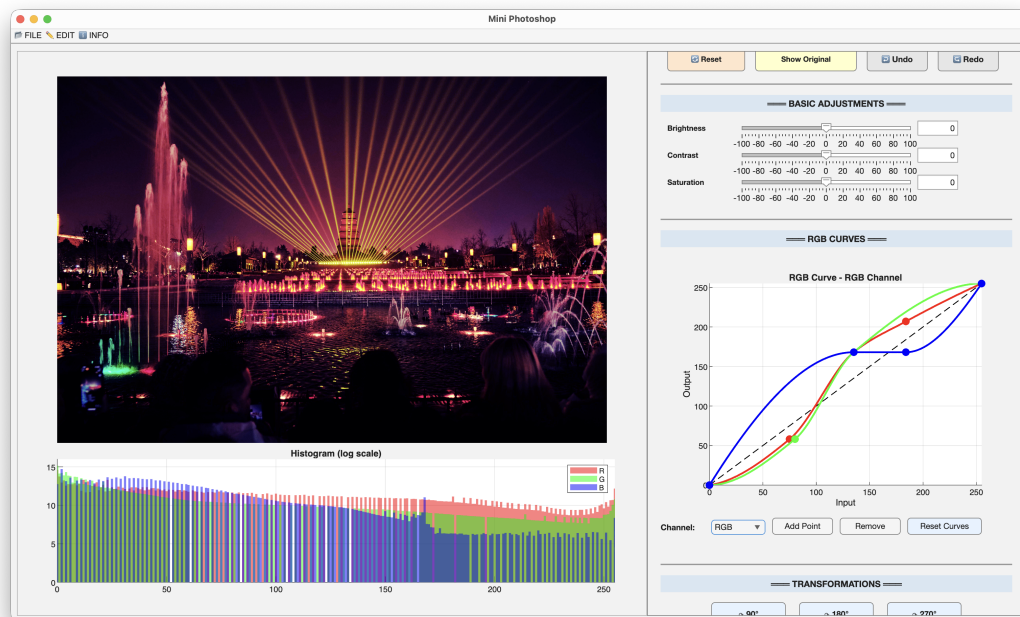


Fig. 4: RGB curve editor interface showing interactive curve manipulation with control points.

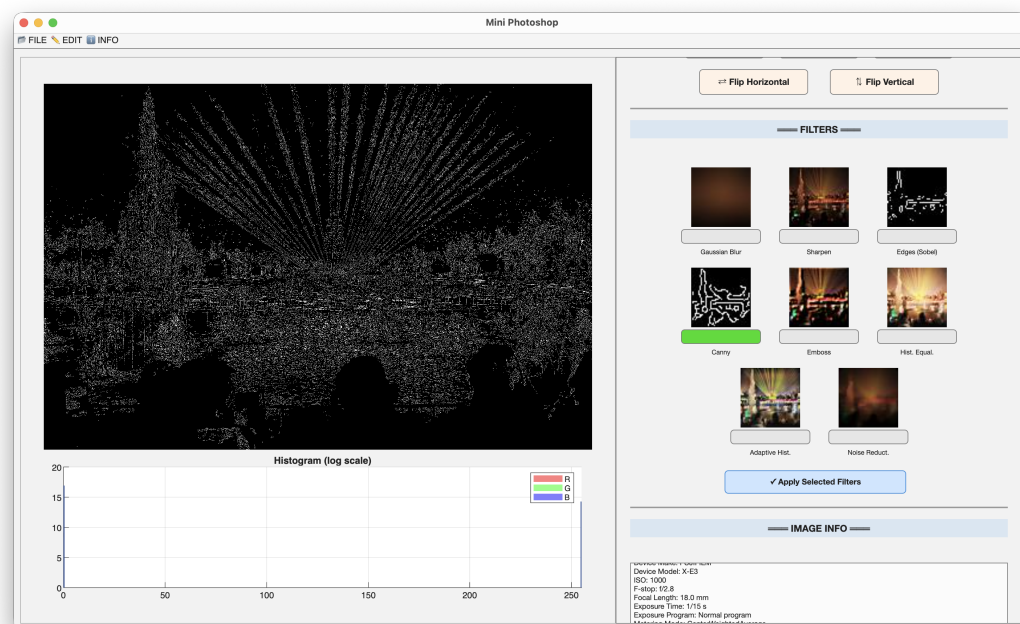
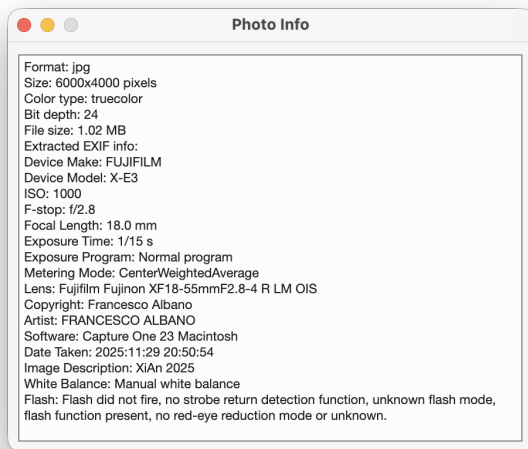
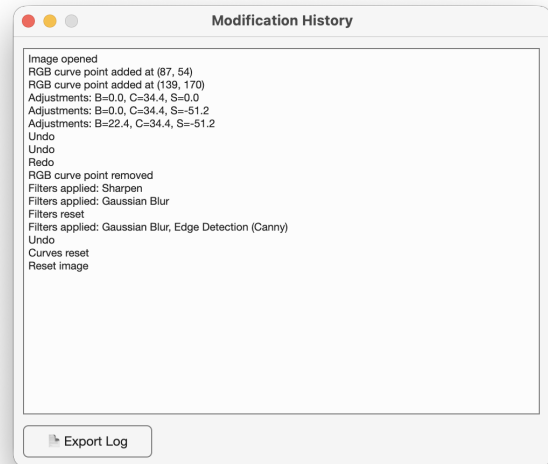


Fig. 5: Example of filter applied to an image.



(a) EXIF metadata panel



(b) History manager panel

Fig. 6: Separate panels for EXIF metadata and history management.

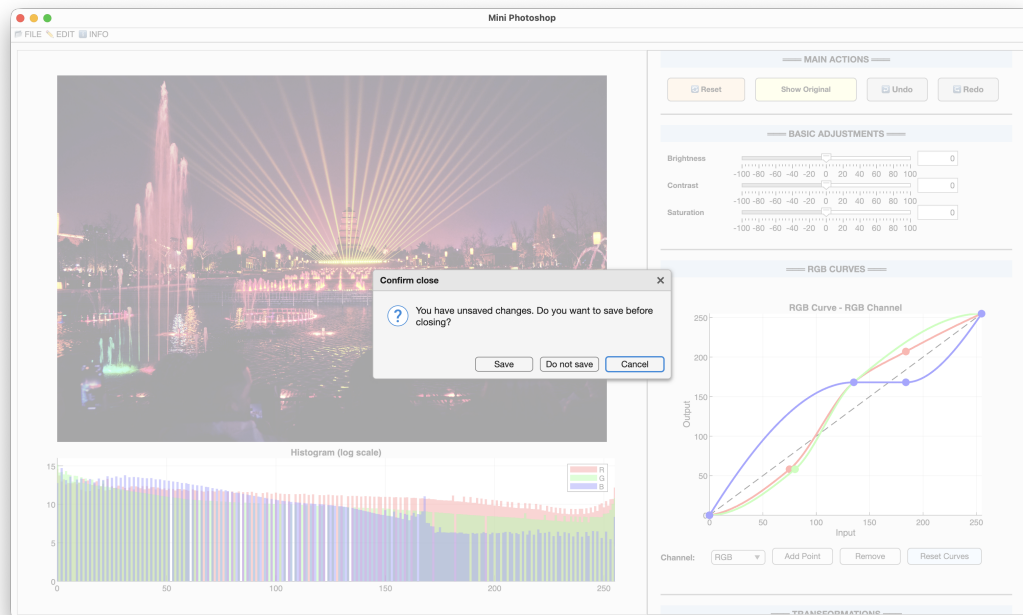


Fig. 7: Unsaved changes confirmation dialog that appears when attempting to exit with unsaved modifications.